Gráfalgoritmusok 1

Horváth Gyula
horvath@inf.elte.hu

Nagyon sok olyan probléma van, amelyek gráf modellben fogalmazhatók meg és a megoldásukat gráfalgoritmusok adják.
Mint ahogy általában is igaz, hogy a modell szint az adatkezelésnek csak az egyik szintje. A hatékony megoldások kifejlesztéséhez megfelelő adatábrázolást, adatszerkezetet kell választani. Az adatszerkezet választását jelentősen befolyásolja, hogy az adatokon milyen műveleteket kell végrehajtani. Ezért fontos, hogy az adatszerkezet választásáról akkor kell dönteni, amikor már ismertek a végzendő műveletek.

1.1. Gráfok modell szintű definíciói

Gráf:$G = (V, E)$, ahol V: a pontok (csúcsok) halmaza, E az élek halmaza
Irányítatlan gráf:$G = (V, E)$
E rendezetlen $\{a, b\}$, $a, b \in V$ párok halmaza.
Irányított gráf:$G = (V, E)$
E rendezett (a, b) párok halmaza; $E \subseteq V \times V$.
Multigráf: $G = (V, E, \mathit{Ind}, \mathit{Érk})$, $\mathit{Ind}, \mathit{Érk} : E \to V$
$\mathit{Ind}(e)$ az e él induló, $\mathit{Érk}(e)$ az érkező pontja.
Címkézett (súlyozott) gráf:$G = (V, E, C)$
$C : E \to \mathit{Címke}$
Minden irányítatlan $G = (V, E)$ gráf olyan irányított gráfnak tekinthető, amelyre teljesül, hogy $(\forall p, q \in V)((p, q) \in E \Rightarrow (q, p) \in E)$.

1.2. Műveletek gráfokon

Jelölések
`KiEl(G, p) = {q ∈ V : (p, q) ∈ E}`
`BeEl(G, p) = {q ∈ V : (q, p) ∈ E}`
`KiFok(G, p) = |Ki(G, p)|`
`BeFok(G, p) = |Be(G, p)|`
A `KiEl(G, p)` halmaz elemeit a `p` *pontszomszédjainak* is nevezzük. Milyen műveleteket akarhatunk gráfokon végezni?

- Pontokszáma
- Élekszáma
- KiFok(p)
- BeFok(p)
- PontBővit(p)
- PontTöröl(p)
- ElBővit(p, q)
- ElTöröl(p, q)
- VanEl(p, q)
- `p` szomszédainak sobavétele: **for q in KiEl(p) do M(p,q)**

Egy adott probléma megoldása esetén általában nincs szükség az összes felsorolt műveletre.

2. Gráfok ábrázolása

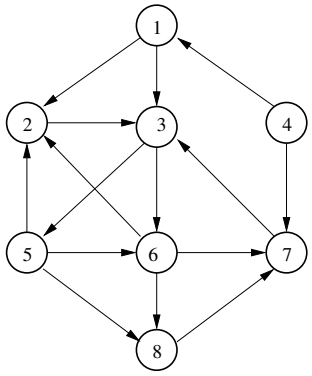
A továbbiakban feltételezzük, hogy a gráf pontjait természetes számokkal azonosítjuk, pontosabban $V \subseteq \{1, \ldots, n\}$ Majd megmutatjuk, hogy hogyan kezelhetők az olyan esetek, probléma szintjén a gráf pontjait nem természetes számokkal azonosítják, hanem például szöveggel. Minden $G = (V, E)$ gráf megadható úgy, hogy minden $p \in V$ pontra megadjuk a $KiEl(p) \subseteq V$ halmazt. Tehát a gráf megadható az $g_E : V \to 2^V$ függvénnyel: $g_E(p) = KiEl(p)$, minden $p \in V$.($2^H = \{R : R \subseteq H\}$).
Minden olyan eszköz, adattípus, adatszerkezet, amely alkalmas `KiEl(p)` halmazok megadására, gráfábrázolási módszert ad.

2.1. Gráfok ábrázolásai

Szempontok az adatszerkezet megválasztásához.

1. Az adott probléma megoldásához ténylegesen mely műveletek szükségesek.
2. Melyek a releváns műveletek, amelyek alapvetően befolyásolják az algoritmus futási idejét.
3. A tárigény az adott probléma esetén.

2.2. Szomszédsági mátrix (Csúcsmátrix)



	1	2	3	4	5	6	7	8
1		1	1					
2			1					
3					1	1		
4	1						1	
5		1				1		1
6		1					1	1
7			1					
8							1	

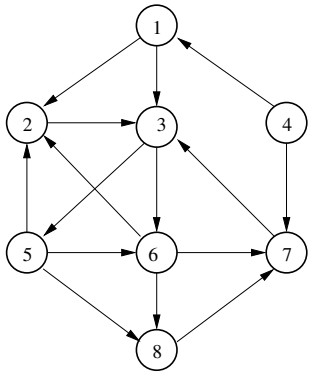
Adatszerkezet

```
//Gráf ábrázolása : Szomszédsági mátrix
#include <bits/stdc++.h>
#define maxP 1000 //pontok max száma
using namespace std;

typedef
    bool Graf[maxP][maxP]; //csúcsmátrix (szomszédsági mátrix)

int main(){
    Graf G;

//gráf előállítása szöveges bemenetből
    int n;// pontok száma
    int m;// élek száma
    int p,q;
    cin>>n>>m;
    for(int x=1;x<=n;x++)// inicializálás
        for(int y=1;y<=n;y++)
            G[x][y]=false;
    for(int i=0;i<m;i++){
        cin>>p>>q;
        G[p][q]=true;
        //G[q][p]=true; //ha irányítatlan
    }
//Élbővít(G,p,q)
    G[p][q]=true;
//Éltöröl(G,p,q)
```

```
    G[p][q]=false;
//Vanél(G,p,q)
    bool van=G[p][q];
//Kifok(G,P)
    int kifok=0;
    for(int x=1;x<=n;x++)
        if(G[p][x]) kifok++;
//Befok(G,p)
    int befok=0;
    for(int x=1;x<=n;x++)
        if(G[x][p]) befok++;
//p Somszédainak sióorbavétele
    for(int q=1;q<=n;q++){
        if(G[p][q]){//p->q él

        }
    }
//Összes él listázása
    for(int p=1;p<=n;p++){
        for(int q=1;q<=n;p++)
            if(G[p][q]){//p->q feldolgozása
                cout<<p<<"->"<<q<<endl;
            }
    }
}
```

Mivel a tárigény négyzetes a pontok számában, ezért a használhatósága korlátozott. Néhány művelet, mint a VanÉl viszont konstans futási idejű.

2.3. Statikus szomszédsági (csúcs) lista



G	1	2	3	4	5	6	7	8	Fok Kifok
1	2	3							2
2	3								1
3	5	6							2
4	1	7							2
5	2	8	6						3
6	7	8	2						3
7	3								1
8	7								1

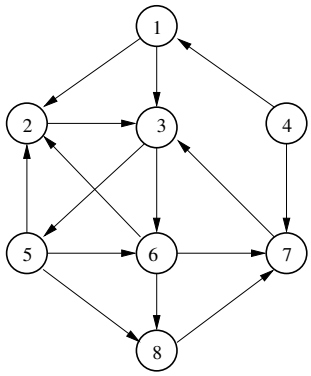
Egyetlen előnye a szomszédsági mátrixszal szemben, hogy a szomszédok sorba-vétele a pontok számával arányos. Hátránya viszont, hogy a VanÉl nem konstans idejű.

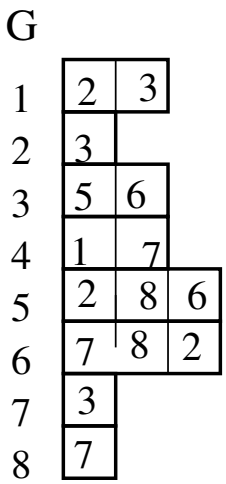
Adatszerkezet

```
//Gráf ábrázolása : csucslista (statikus szomszédsági lista)
#include <bits/stdc++.h>
#define maxP 1000 //pontok max száma
using namespace std;
typedef
    int Graf[maxP][maxP]; //csúcsmátrix (szomszédsági mátrix)

int main(){
    Graf G;
    int Fok[maxP];

//gráf előállítása szöveges bemenetből
    int n;// pontok száma
    int m;// élek száma
    int p,q;
    cin>>n>>m;
    for(int x=1;x<=n;x++){// inicializálás
        Fok[x]=0;
    }
    for(int i=0;i<m;i++){
        cin>>p>>q;
        Fok[p]++;
        G[p][Fok[p]]=q;
        //G[q][++Fok[q]]=p; //ha irányítatlan
    }
```

```
//Élbővít(G,p,q)
   G[p][++Fok[p]]=q;
//Éltöröl(G,p,q)
//q törlés a G[p]  sorból
//Vanél(G,p,q)
   bool van=false;
   for(int i=1;i<=Fok[p]; i++){
      if(G[p][i]==q){ van=true; break; }
   }
//Kifok(G,P)
   int kifok=Fok[p];
//Befok(G,p)
//ahány sorban szerepel p
//p Somszédainak sióorbavétele
   for(int i=1;i<=Fok[p];i++){
       q=G[p][i];
       //p->q él
   }
//Összes él listázása
   for(int p=1;p<=n;p++){
       for(int i=1;i<=Fok[p];i++)
           if(G[p][i]){//p->q feldolgozása
               cout<<p<<"->"<<G[p][i]<<endl;
           }
   }
}
```

2.4. Dinamikus szomszédsági (csúcs) lista





A leggyakrabban alkalmazott ábrázolási mód. Ha külön nem jelezzük, akkor a továbbiakban ezt tekintjük standard módszernek.
Előnye, hogy a tárigény az élek számával arányos, és a szomszédok sorba-vétele is a pontok számával arányos.

Adatszerkezet

```
1  //Gráf ábrázolása : dinamikus csucslista (dinamikus szomszédsági l
2  #include <bits/stdc++.h>
3  #define maxP 1000 //pontok max száma
4  using namespace std;
5  typedef
6      vector<int> Graf[maxP];//csúcslista
7  // p csomszédai a G[p] vektor (tomb) elemei
8  int main(){
9      Graf G;
10
11 //gráf előállítása szöveges bemenetből
12     int n;// pontok száma
13     int m;// élek száma
14     int p,q;
15     cin>>n>>m;
16     for(int i=0;i<m;i++){
17         cin>>p>>q;
18         G[p].push_back(q);
19         //G[q].push_back(p) //ha irányítatlan
20     }
21
22 //Élbővít(G,p,q)
```

```
    G[p].push_back(q);
//Éltöröl(G,p,q)
//q törkése a G[p] sorból
//Vanél(G,p,q)
    bool van=false;
    for(int x : G[p]){
      if(x==q){ van=true; break; }
    }
//Kifok(G,P)
    G[p].size();
//Befok(G,p)
    int befok=0;
    for(int q=1; q<=n; q++)
        for(int x:G[q])
        if (x==p) befok++;
// Somszédok bejárása
    for(auto q:G[p]){
        //p->q él feldolgozása
    }
//Összes él listázása
    for(int p=1; p<=n; p++){
        for(int q:G[p])
            cout<<p<<"->"<<q<<endl;
    }
}
```

2.5. Éllista

Az élek halmazát az élek, mint számpárok sorozatával adjuk meg.
Ez a módszer akkor használható eredményesen, ha az algoritmus azt igényli, hogy az éleket vegyük megfelelő sorrendben.

Adatszerkezet

```
//Gráf ábrázolása : (dinamikus) éllista
#include <bits/stdc++.h>
using namespace std;

typedef
    vector<pair<int,int>> Graf;//éllista

int main(){
    Graf G;

//gráf előállítása szöveges bemenetből
    int n;// pontok száma
    int m;// élek száma
    int p,q;
    cin>>n>>m;
    for(int i=0;i<m;i++){
        cin>>p>>q;
        G.push_back({p,q});
        //G.push_back({q,p}) //ha irányítatlan
    }

//Élbővít(G,p,q)
    G.push_back({p,q});
//Éltöröl(G,p,q)
//a {p,q} pár törlése a G vektorból
//Vanél(G,p,q)
    bool van=false;
    for(auto xy : G){
      if(xy.first==p && xy.second==q){ van=true; break; }
    }
//Kifok(G,P)
    int kifok=0;
    for(auto xy : G){
```

```
        if(xy.first==p){ kifok++; }
    }
//Befok(G,p)
    int befok=0;
    for(auto xy : G){
        if(xy.second==p){ befok++; }
    }
// Somszédok bejárása; nagyon lassú!
    for(auto xy : G)
        if(xy.first==p){ q=xy.second; }

//Összes él listázása
    for(auto xy : G){
        p=xy.first; q=xy.second;
        cout<<p<<"->"<<q<<endl;
    }
}
```

2.6. Ábrázolás a szomszédok halmazával

Ha olyaan feladatot kell megoldani, ahol a pontok száma `N` lehetővé teszi, `N`-elemű tömb foglalását, és VanÉl műveleteket is alkalmaznk kell, akkor a szomszédok `set<int>` típussal ábrázolva, minden VanÉl futási ideje `O(log(n))` lesz. Ugyancsak `O(log(n))` időben lehet élet bővíteni és törölni.

Adatszerkezet

```
1  #define maxN 10001
2
3  using namespace std;
4  typedef
5  set<int> Graf[maxN];
6
7  void Beolvas(Graf& G, int &n){
8    //gráf előállítása szöveges bemenetből
9    int m;// pontok és élek száma
10   int p,q;
11   cin>>n>>m;
12   for(int i=0;i<m;i++){
13      cin>>p>>q;
14      G[p].insert(q);
15      //G.Elek[q].insert(p);//irányítatlan esetben
16   }
17 }
18
19 bool Vanel(Graf& G, int p, int q){
20   //  auto it=G[p].find(q);
21   //ha q eleme a G[p] halmaznak, akkor *it=q
22   return G[p].find(q)!=G[p].end();
23 };
24 void Elbovit(Graf& G, int p, int q){
25   G[p].insert(q);
26 }
27 void Eltotol(Graf& G, int p, int q){
28   G[p].erase(q);
29 }
30 int Kifok(Graf& G, int p){
31   return G[p].size();
32 }
```

2.7. Ábrázolás koordináta kompresszióval

Ha a pontok azonosítására használt `PontTip` adattípuson értelmezett lineáris rendezési reláció, például string, vagy koordináta számpárok, akkor a bemenetből előállíthatunk egy sorszámosítást, és az algoritmust a standard szomszédsági listás ábrázolással valósíthatjuk meg. Az eredmény szükség esetén a bemenetnek megfelelő azonosítással adhatjuk meg.

A sorszám hozzárendelést megoldhatjuk a `map<PontTip, int>` adattípus használatával.

Adatszerkezet

```
1  typedef string PontTip;
2
3  struct Graf{
4    vector<PontTip> V;//a gráf pontjainak halmaza
5    vector<vector<int>> E;
6    map<PontTip, int> Nev_szam;//p sorszáma=Nev_szam[p]
7    int Sorszam(PontTip p);
8  };
9
10 int Graf:: Sorszam(PontTip p){
11   auto it=Nev_szam.find(p);
12   if(it!=Nev_szam.end())
13     return it->second;  //p már szerepelt, sorszáma
14                         //Nev_szam[p]=it->second
15   int ps=V.size();
16   Nev_szam.insert({p, ps});
17   //Nev_szam[p]=ps;
18   V.push_back(p);
19   vector<int> ures;
20   E.push_back(ures);//üres sor az új p számára
21   return ps;
22 }
23
24 void Beolvas(Graf& G){
25   //gráf előállítása szöveges bemenetből
26   int m;// élek száma
27   PontTip p,q;
28   string s;
29   cin>>m;
30   getline(cin,p);
31   for(int i=0;i<m;i++){
32     cin>>p>>q;
```

```
33        int ps=G.Sorszam(p);
34        int qs=G.Sorszam(q);
35       G.E[ps].push_back(qs);
36     }
37 }
38 int main(){
39     Graf G;
40     Beolvas(G);
41     cout<<"V:"<<endl;
42     for(PontTip x : G.V) cout<<x<<","; cout<<endl;
43     cout<<"Élek:"<<endl;
44     for(int x=0; x<G.V.size(); x++){
45        PontTip xn=G.V[x];
46        cout<<xn<<"->";
47        for(int y:G.E[x])cout<<G.V[y]<<","; cout<<endl;
48     }
49 }
```

2.8. Univerzális gráfábrázolás

Minden `(V, E)` gráf megadható oly módon, hogy minden `p` $\in$ `V` pontra megadjuk a `p` szomszédainak halmazát. Ha a pontokat olyan `PontTip` típusú értékekkel azonosítjuk, amelyen értelmezett lineáris rendezés (pl. int, string), akkor a gráf ábrázolható a `map < Tpont, set < PontTip >>` típussal.
A legtöbb művelet futási ideje `O(log(n))` legrosszabb esetben is, ha a pontok száma `n`.

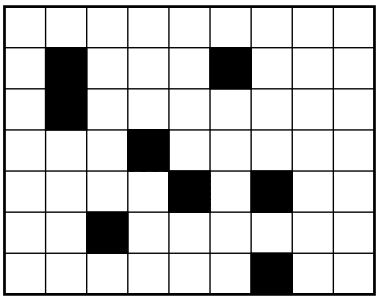
Adatszerkezet

```
1  typedef string PontTip;
2  struct Graf{
3    set<PontTip> V;//a gráf pontjainak halmaza
4    map<PontTip, set<PontTip>> E;
5    //p szomszédainak halmaza E[p]
6  };
7
8  void Beolvas(Graf& G){
9    //A G gráf előállítása szöveges bemenetből
10   //PontTip=string
11   int m;
12   PontTip p,q;
13   cin>>m;
14   getline(cin,p);
15   for(int i=0;i<m;i++){
16     cin>>p>>q;
17     G.V.insert(p); G.V.insert(q);
18     G.E[p].insert(q);
19   }
20 }
21 int Pontokszama(Graf& G){
22   return G.V.size();
23 }
24 void Pontbovit(Graf& G, PontTip p){
25   G.V.insert(p);
26 }
27 void PontTorol(Graf& G, PontTip p){
28   G.V.erase(p);
29   G.E.erase(p);
30   for(auto q:G.V)
31   G.E[q].erase(p);
32 }
33 bool Vanel(Graf& G, PontTip p, PontTip q){
```

```
  //  auto it=G.E[p].find(q);
  //ha q eleme a G.E[p] halmaznak, akkor *it=q
  return G.E[p].find(q)!=G.E[p].end();
};
void Elbovit(Graf& G, PontTip p, PontTip q){
  G.E[p].insert(q);
}
void Eltotol(Graf& G, PontTip p, PontTip q){
  G.E[p].erase(q);
}
int Kifok(Graf& G, PontTip p){
  return G.E[p].size();
}
int Befok(Graf& G, PontTip p){
  int befok=0;
  for(auto q : G.V)
  if(G.E[q].find(p)!=G.E[q].end()) befok++;
  return befok;
}

int main(){
  Graf G;
  Beolvas(G);
  cout<<Pontokszama(G)<<endl;
  cout<<"G pontjai:\n";
  for(auto x:G.V) cout<<x<<","; cout<<endl;
  cout<<"G élei:\n";
  for(auto p:G.V){
    cout<<p<<"->:";
    for(auto q:G.E[p]) cout<<q<<","; cout<<endl;
  }
}
```

2.9. Számított gráf

Az élek halmazát explicite nem tároljuk, mert van olyan számítási eljárás, amely bármely két `p, q` $\in$ `V`-re kiszámítja `VanEl(p, q)`-t. Vagy, van olyan számítási eljárás, amely minden `p` $\in$ `V`-re kigenerálja a szomszédok, azaz a `Ki(G, p)` halmaz elemeit.
Ilyen például egy labirintus gráfja. A gráf pontjai a cellák, az (x_1, y_1) cellából akkor és csak akkor van él az (x_2, y_2) cellába, ha (x_2, y_2) a négy szomszéd valamelyik cellája, és nem fal.



1. ábra. Labirintus

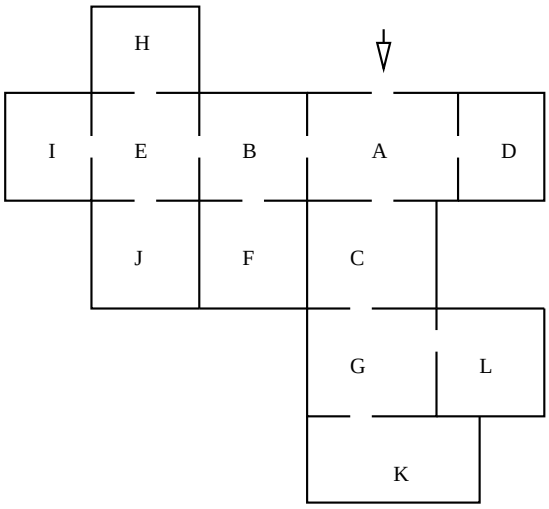
3. Mélységi bejárás

Gráf bejáráson olyan eljárást értünk, amely adott kiindulási pontból indulva, a gráf éleit szerint haladva sorba-veszi az így elérhető pontokat.
Alapvetően két bejárási stratégia különböztethető meg. A szélességi és a mélységi. A szélességi bejárás az aktuális pont minden szomszédját vizsgálja, és ha még nem elért, akkor beteszi az aktuális pontok halmazába, amelyből általában érkezési idő szerint választ következő aktuális pontot.
A mélységi stratégia alkalmazásakor egyetlen aktív pont van, az aktuális. Az aktuális pont bármely olyan szomszéd pontját választhatjuk aktuálisnak, amelyben még nem jártunk. Ha nincs ilyen, akkor visszamegyünk oda, ahonnan az aktuális pontba léptünk.

3.1. Képtár látogatás

Egy sok teremből álló képtárban teszünk látogatást. Tudjuk, hogy a főbejárattól a képtár bármely termébe pontosan egy útvonalon keresztül lehet eljutni.

Adjunk olyan bejárási stratégiát, amely biztosítja, hogy minden terembe eljutunk (minden képet pontosan egyszer nézünk meg)!

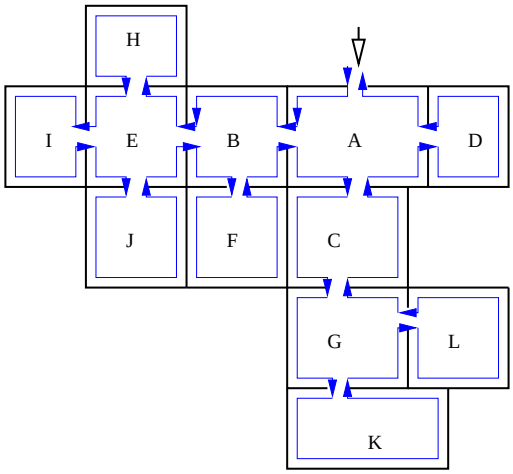


2. ábra

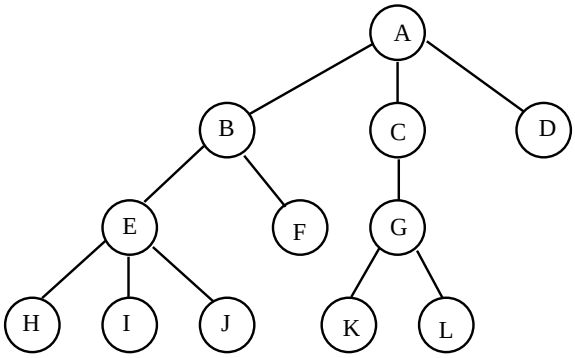
Megoldás

A fal mellett mindig jobbra haladjunk.

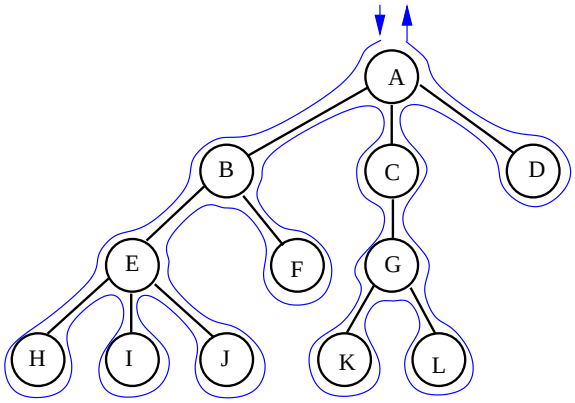
Tekintsük azt a `G = (V, E)` gráfot, amelynek pontjai (csúcsai) a képtár termei, (V=A,B,C,D,E,F,G,H,I,J,K,L), `(p, q)` $\in$ `E` akkor és csak akkor, ha `p` teremből nyílik ajtó a `q` terembe.
Ez a gráf fa.



3. ábra



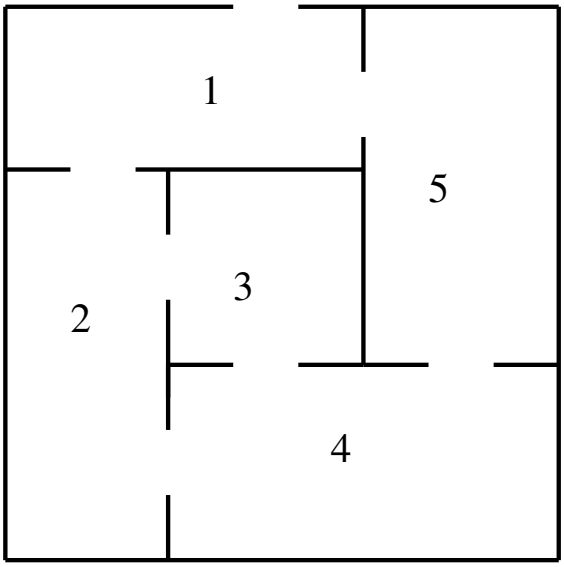
4. ábra



5. ábra

3.2. Feladat : Második képtárlátogatás

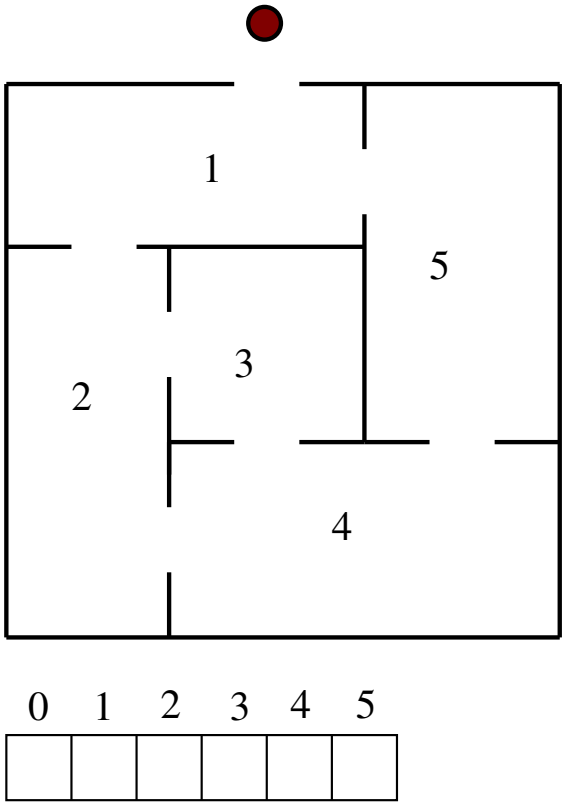
Olyan képtárban teszünk látogatást ahol van olyan terem, amelybe többféle úton is el lehet jutni a bejárattól. Adjunk olyan bejárási stratégiát, amely biztosítja, hogy minden terembe eljutunk, és visszaérünk a bejárathoz.!
Minden teremben kirakták a terem sorszámát és minden ajtón rajta van, hogy melyik terembe vezet.



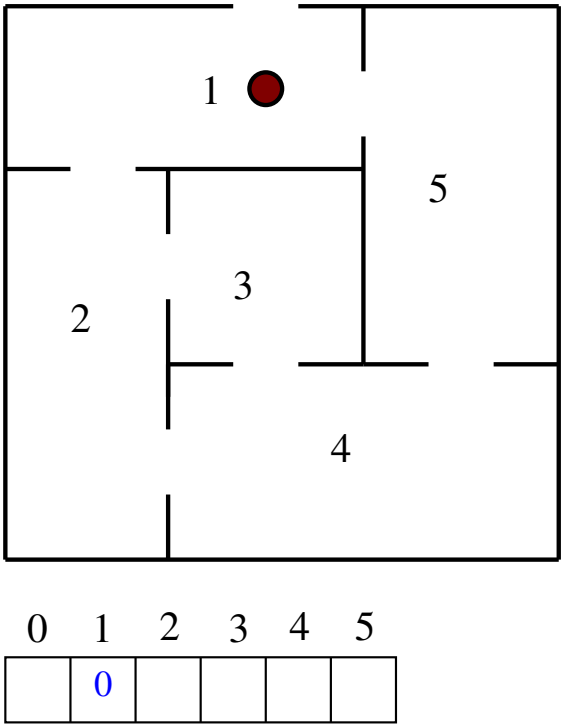
6. ábra. A képtár termeinek alaprajza

- Mit kell tudnunk?

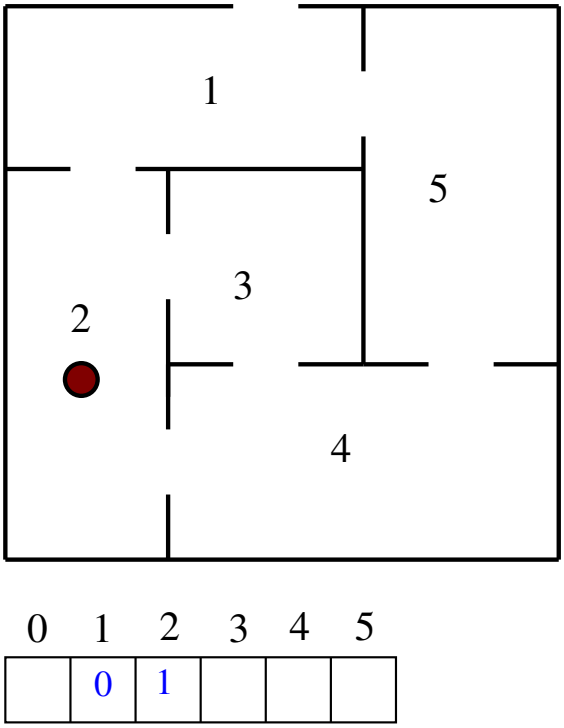
1. Jártunk-e már az adott teremben?
2. Melyik teremből léptünk először az adott terembe?



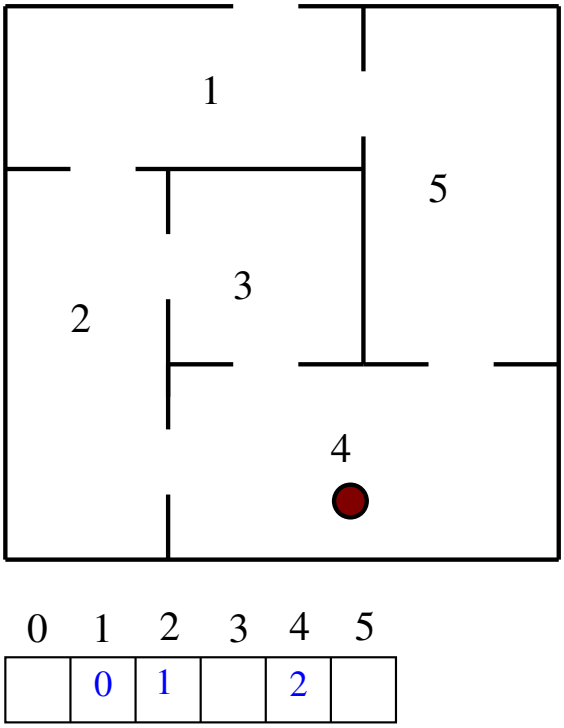
7. ábra



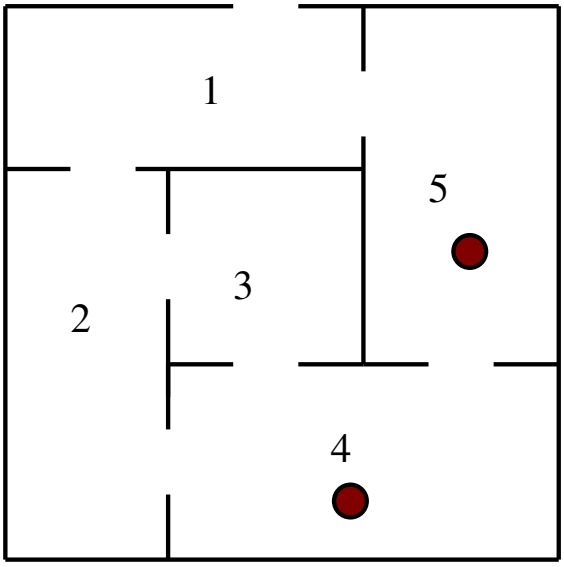
8. ábra



9. ábra

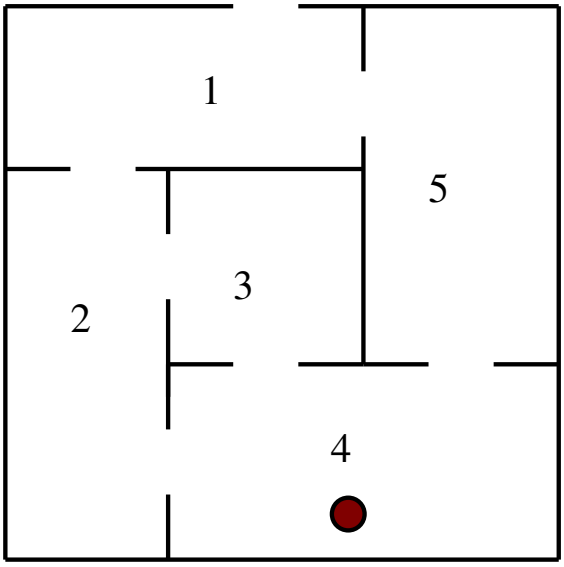


10. ábra



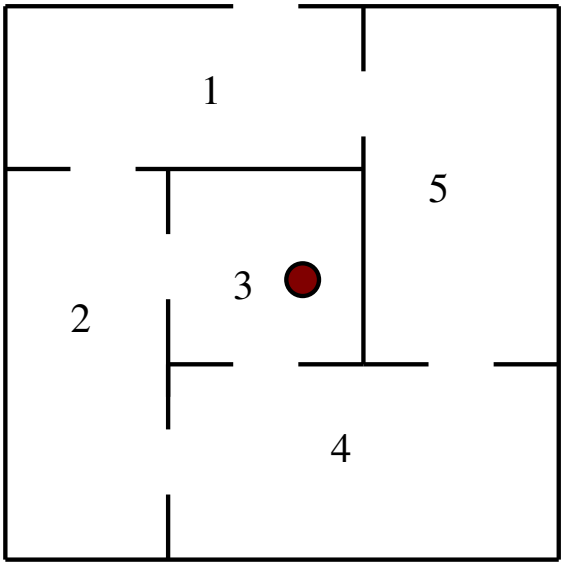
0	1	2	3	4	5
	0	1		2	4

11. ábra



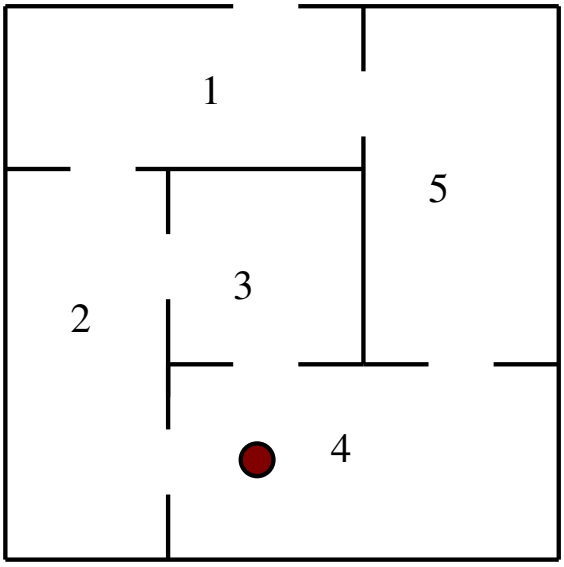
0	1	2	3	4	5
	0	1		2	4

12. ábra



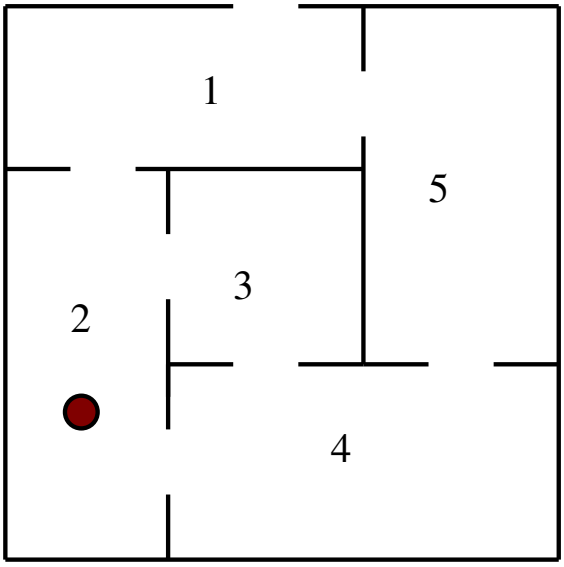
0	1	2	3	4	5
	0	1	4	2	4

13. ábra



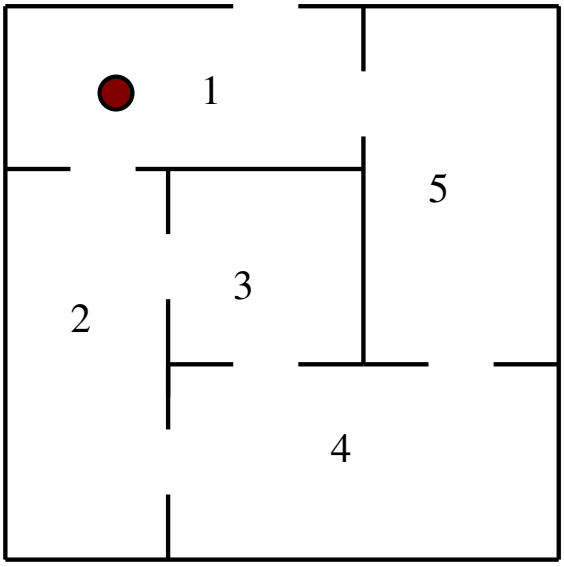
0	1	2	3	4	5
	0	1	4	2	4

14. ábra



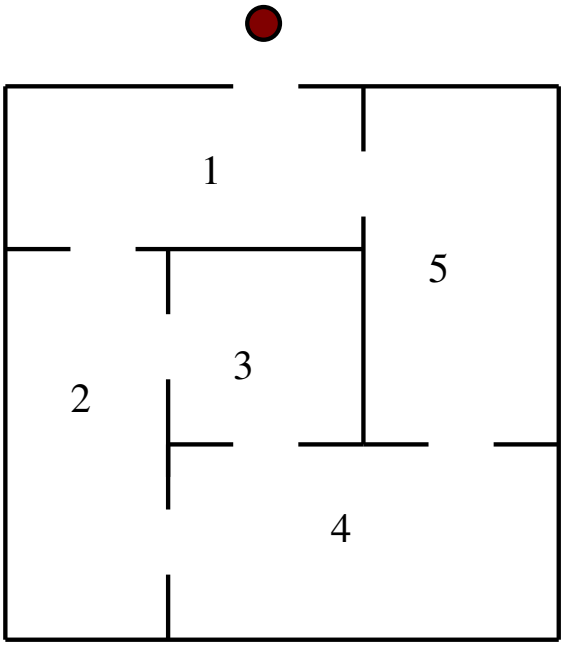
0	1	2	3	4	5
	0	1	4	2	4

15. ábra



0	1	2	3	4	5
	0	1	4	2	4

16. ábra



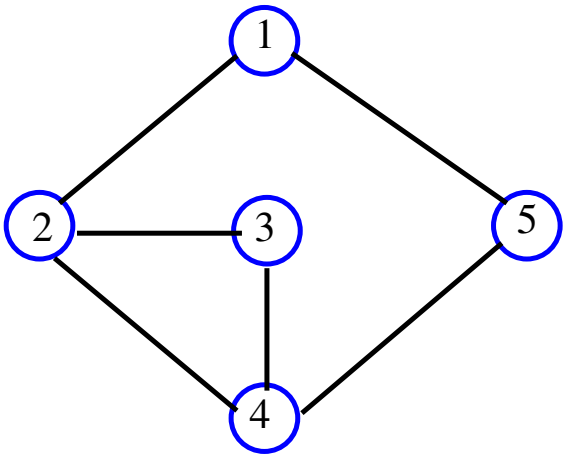
0	1	2	3	4	5
	0	1	4	2	4

17. ábra

Minden `p` teremre `Honnan(p)` legyen annak a teremnek a száma, amelyből először lépünk a `p` terembe. Kezdetben legyen minden `p`-re `Honnan(p) = 0`. Így ha a `p` teremben vagyunk, és a `q` terembe vezet ajtó, a `Honnan(q)` értéke alapján el tudjuk dönteni, hogy jártunk-e `q`-ban.

Az elvi algoritmus:

```
at = 1;              // az aktuális terem
Honnan(1) = 0; //az utcáról léptünk a főbejárat termébe
while(at != 0){//amíg vissza nem értünk a bejárat/kijárathoz
    if (at-nek van q benemjárt szomszédos terme){
        Honnan(q) = at;
        at = q;
    }else
        at = Honnan(at);
}
```



18. ábra. A képtár gráfja

A mélységi bejárás stratégiája egyszerűen megvalósítható rekurzióval, mert a visszalépés automatikusan megvalósul az eljáráspéldány terminálásával.
A mélységi bejárásnak számos változata van, annak megfelelően, hogy milyen konkrét probléma megoldására használjuk.

3.3. Mélységi bejárás: elérhető pontok

```
1  typedef
2  vector<int> Graf[maxN];
3  Graf G;
4  int n;
5  bool Elert[maxN];
6
7  void MelyBejar1(int p){
8    //Elérhetők meghatározása
9    //Globális: G, Elert
10   Elert[p]=true;
11   for (int q:G[p])//p->q él
12   if (!Elert[q])//q-ban még nem jártunk
13   MelyBejar1(q);
14 }
```

3.4. Mélységi bejárás: feszítőfa építés

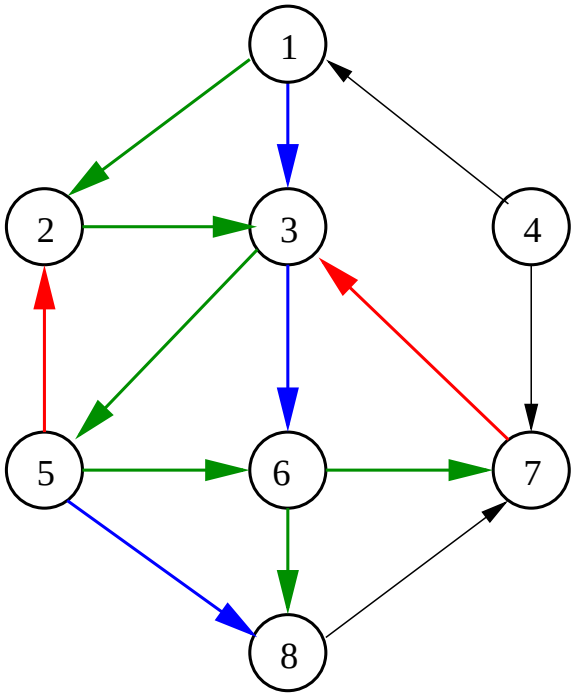
```
typedef vector<int> Graf[maxN];
Graf G;
int n;
int FeFa[maxN];

void MelyBejar2(int p){
  //Feszítőfa építése
  //Globális: G, FeFa,
  for (int q:G[p])//p->q él
  if (FeFa[q]<0){//q-ban még nem jártunk
    FeFa[q]=p;
    MelyBejar2(q);
  }
}
void Beolvas(){
  //Globális: G, n
  int m,x,y;
  cin>>n>>m;
  for(int i=0;i<m;i++){
    cin>>x>>y;
    G[x].push_back(y);
  }
}
int main(){
  Beolvas();
  for(int x=1;x<=n;x++) FeFa[x]=-1;
  FeFa[1]=0;
  MelyBejar2(1);
  //egy út kiíratása 1-ből n-be:
  vector<int> Ut;
  int p=n;
  while(p!=0){
    Ut.push_back(p);
    p=FeFa[p];
  }
  for(int i=Ut.size()-1; i>=0; i--) cout<<Ut[i]<<",";
  cout<<endl;
}
```

3.5. Mélységi bejárás: élek színezése

```
typedef
  vector<int> Graf[maxN];
Graf G;
enum Paletta {Feher, Szurke, Fekete};
int FeFa[maxN];
Paletta Szin[maxN];
void MelyBejar3(int p){
  //Élek színezése
  //Globális: G, Szin
  Szin[p]=Szurke;
  for (int q:G[p]){//p->q él
    if (Szin[q]==Feher){//q-ban még nem jártunk
      MelyBejar3(q);
    }else if(Szin[q]==Szurke){//visszaél
    }
  }
  Szin[p]=Fekete;
}
```

3.6. Élek osztályozása

1. **Faél:** `(p, q)` $\in$ `V` faél, ha bekerül a feszítőfa élei közé, azaz `FeFa(q) = p`.
2. **Visszaél:** `(p, q)` $\in$ `V` visszaél, ha `p` leszármazottja `q`-nek a feszítőfában.
3. **Előreél:** `(p, q)` $\in$ `V` előre-él, ha `q` leszármazottja `p`-nek a feszítőfában és nem faél.
4. **Keresztél:** Minden más esetben `(p, q)` $\in$ `V` keresztél.



19. ábra. Élek osztályozása: zöld: faél, piros: visszaél, kék: előreél, fekete: keresztél

Állítás

1. `(p, q)` $\in$ `V` akkor és csak akkor faél, ha az `p` $\rightarrow$ `q` él vizsgálatakor `q` színe fehér.
2. `(p, q)` $\in$ `V` akkor és csak akkor visszaél ha az `p` $\rightarrow$ `q` él vizsgálatakor `v` színe szürke.
3. `(p, q)` $\in$ `V` akkor és csak akkor előre-él ha az `p` $\rightarrow$ `q` él vizsgálatakor `q` színe fekete és `p`-t korábban értük el, mint `q`-t.
4. `(p, q)` $\in$ `V` akkor és csak akkor keresztél ha az `p` $\rightarrow$ `q` él vizsgálatakor `q` színe fekete és `q`-t korábban értük el, mint `p`-t.

Állítás

A `G = (G, V)` irányított vagy irányítatlan gráfban akkor és csak akkor van kör, ha a mélységi bejárás során van visszaél.

3.7. Mélységi bejárás: elérési és elhagyási idők számítása

```
int D[maxN];//elérési idő
int F[maxN];//elhagyási idő
int ido=0;  //bejárási globális idő

void MelyBejar3(int p){
//Elérési és elhagyási idok számítása
  //Globális: G, FeFa, D, F, ido
  //D[p] kezdeti értéke 0
  ido++;
  D[p]=ido;
  for (int q:G[p]){//p->q él
    if (D[q]==0){////q-ban még nem jártunk
      FeFa[q]=p;
      MelyBejar3(q);
    }
  }
  ido++;
  F[p]=ido;
}
```

Állítás

Bármely mélységi bejárás során kiszámított `D`, `F`, `FeFa` értékekre teljesül, hogy bármely két különböző `p` és `q` pont esetén az alábbi három feltétel közül pontosan az egyik teljesül:

1. `F[p] < D[q]`

2. `F[q] < D[p]`

3. `D[p] < D[q] < F[q] < F[p]`

Másképpen fogalmazva, a `[D[p], F[p]]` intervallumok egymásba skatulyázottak.

Állítás

`q` akkor és csak akkor leszármazottja `q`-nak a feszítőfában, ha

$$\texttt{D[p]} < \texttt{D[q]} < \texttt{F[q]} < \texttt{F[p]}$$

3.1. tétel. *Ha* `G` *irányítatlan gráf, akkor bármely mélységi keresésre minden* $\texttt{p} - \texttt{q}$ *él vagy fa-él, vagy vissza-él.*

Bizonyítás

Tfh. `p`-t előbb értük el, mint `q`-t. Mivel $\texttt{q} \in \texttt{KiEl(G, p)}$, ezért valamikor végrehajtódik a $\texttt{p} \to \texttt{q}$ él vizsgálta, és ekkor `Szin(p) = Szurke`. Milyen lehet `q` színe ekkor?

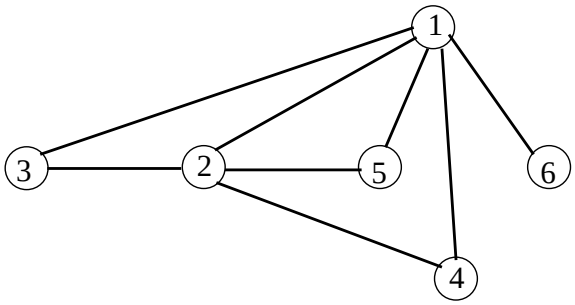
1. `Szin(q) = Feher`: Ekkor `(p, q)` faél lesz.

2. `Szin(q) = Szurke`: Ekkor `(p, q)` visszaél lesz.

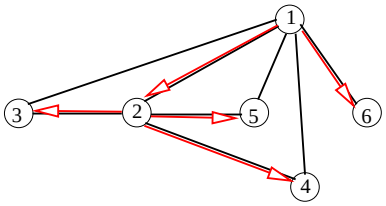
3. `Szin(q) = Fekete`: Nem lehet, mert ez azt jelentené, hogy `q`-t már korábban elértük és az összes belőle kiinduló élet, így p-t is vizsgáltuk.

3.8. Utcaseprő

A város elhatározta, hogy minden nap végigmegy a város utcáin az egyetlen utcaseprő gépe. A város minden utcája kétirányú, és az utcaseprőnek is be kell tartania a közlekedési szabályokat. Adjunk olyan útvonalat, amelyen haladva az utcaseprő minden utcában pontosan egyszer halad végig mindkét irányban.



20. ábra



21. ábra. Mélységi feszítőfa alapján az útvonal: 1-2-3-1-3-2-4-1-4-2-5-1-5-2-1-6-1

```
typedef
    vector<int> Graf[maxN];
enum Paletta {Feher, Szurke, Fekete};
Graf G;
Paletta Szin[maxN];
int n;

void MelyBejar(int apa, int p){
//Global: G,Szin
    Szin[p]=Szurke;
    cout<<p<<" ";
    for (int q:G[p]){
        if (Szin[q]==Feher){
            MelyBejar(p,q);
            cout<<p<<" ";
        }else if(q!=apa && Szin[q]==Szurke){//visszaél
            cout<<q<<" "<<p<<" ";
        }
    }
    Szin[p]=Fekete;
}
```

```
22 void Beolvas(){
23 //Global: G,n,
24     int m,p,q;
25     cin>>n>>m;
26     for (int i=0;i<m;i++){
27         cin>>p>>q;
28         G[p].push_back(q);
29         G[q].push_back(p);//irányítatlan a gráf
30     }
31 }
32 int main(){
33     Beolvas();
34     for(int i=1;i<=n;i++)
35         Szin[i]=Feher;
36     MelyBejar(0,1);
37     cout<<endl;
38   return 0;
39 }
```

4. Euler-út, Euler-kör

4.1. definíció. *A* `G = (V, E)` *(irányított vagy irányítatlan) gráfban Euler-séta olyan séta, amely a gráf minden élét pontosan egyszer tartalmazza.*

4.2. definíció. *A* `G = (V, E)` *(irányított vagy irányítatlan) gráfban Euler-kör olyan séta, amely a gráf minden élét pontosan egyszer tartalmazza, beleértve a séta utolsó pontjából az első pontba vezető élet is.*

4.3. tétel. *A* `G = (V, E)` *irányított gráfban akkor és csak akkor van Euler-kör, ha* `G` *erősen összefüggő, és*

$$(\forall \mathtt{v} \in \mathtt{V})\,(\mathtt{KiFok(v)} = \mathtt{BeFok(v)}) \tag{1}$$

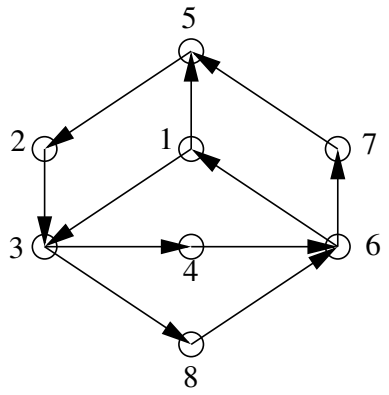
4.4. tétel. *A* `G = (V, E)` *irányított gráfban akkor és csak akkor van Euler-séta, ha van benne Euler-kör, vagy pontosan egy olyan* $\mathtt{a} \in \mathtt{V}$ *és* $\mathtt{b} \in \mathtt{V}$ *pont van, hogy*

$$\mathtt{KiFok(a)} = \mathtt{BeFok(a)} + 1 \tag{2}$$

$$\mathtt{KiFok(b)} + 1 = \mathtt{BeFok(b)} \tag{3}$$

$$(\forall \mathtt{v} \in \mathtt{V})\,(\mathtt{v} \neq \mathtt{a} \wedge \mathtt{v} \neq \mathtt{b} \Rightarrow \mathtt{KiFok(v)} = \mathtt{BeFok(v)}) \tag{4}$$

és `G` *bármely pontja elérhető* `a`*-ból.*



22. ábra. Euler séta: 1 5 2 3 8 6 1 3 4 6 7 5

4.5. tétel. *A* `G = (V, E)` *irányítatlan gráfban akkor és csak akkor van Euler-kör, ha összefüggő, és*

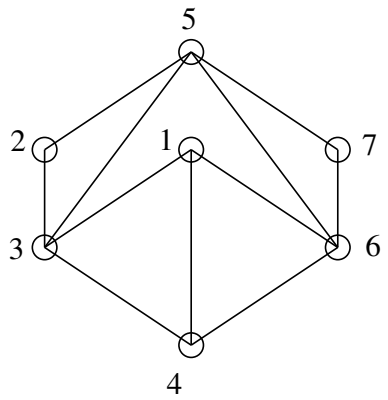
$$(\forall \mathtt{v} \in \mathtt{V})\,(\mathtt{Fok(v)}\ \textit{páros}) \tag{5}$$

4.6. tétel. *A* $\mathtt{G} = (\mathtt{V}, \mathtt{E})$ *irányítatlan gráfban akkor és csak akkor van Euler-séta, ha van benne Euler-kör, vagy összefüggő és pontosan egy olyan* $\mathtt{a} \in \mathtt{V}$ *és* $\mathtt{b} \in \mathtt{V}$, $\mathtt{a} \neq \mathtt{b}$ *pont van, hogy*

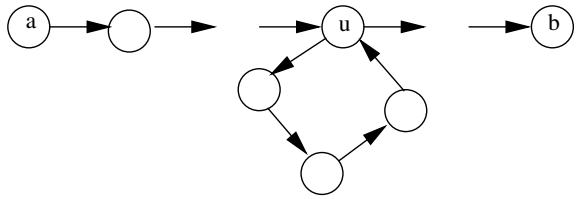
$$\mathtt{Fok}(\mathtt{a}) \textit{ páratlan} \tag{6}$$

$$\mathtt{Fok}(\mathtt{b}) \textit{ páratlan} \tag{7}$$

$$(\forall \mathtt{v} \in \mathtt{V})\ (\mathtt{v} \neq \mathtt{a} \wedge \mathtt{v} \neq \mathtt{b} \Rightarrow \mathtt{Fok}(\mathtt{v}) \textit{ páros}) \tag{8}$$



23. ábra. Euler séta:1 3 4 6 5 3 2 5 7 6 1 4



24. ábra. Vakséta

Tegyük fel, hogy a G irányított gráfra teljesül a az Euler-kör feltétele. Tekintsünk egy tetszőleges vak sétát az a pontból kiindulva. A vak séta azt jelenti, hogy tetszőlegesen haladhat, azt az élet, amelyen áthalad törli, mindaddig halad, amíg olyan b pontba nem ér, amelyből már nem indul ki. Ekkor a = b mert minden pont kifoka megegyezik befokával. Legyen a vaksétában u az utolsó olyan pont, amelyből indul ki be-nemjárt él. Ekkor a sétában az u utáni rész a kialakítandó séta végleges része lesz. Az élek törlése után keletkező gráfra is teljesül a feltétel. Végezzünk egy vaksétát az u pontból kiindulva és illesszük be az eddigi sétába. Mindezt addig ismételjük, amíg van benemjárt él.Ha a vaksétát veremben tároljuk, akkor az u pontot úgy találjuk meg, hogy a pontokat mindaddig kivesszük, amíg a ponthoz nem tartozik benemjárt él.

4.1. Euler séta irányított gráfban.

```
1  void EulerSeta(Graf& G, int a, vector<int>& Seta){
2    //Kifok(a)=Befok(a)+1
3    //van egy b: Befok(b)=Kifok(b)+1
4    stack<int> V;//a vakséta
5    V.push(a);
6    while(!V.empty()){
7      int p=V.top();
8      if(G[p].size()==0){
9        V.pop();
10       Seta.push_back(p);
11     }else{
12       int q=G[p].back(); G[p].pop_back();
13       V.push(q);
14     }
15   }
16   reverse(Seta.begin(), Seta.end());
17 }
```

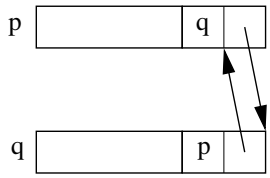
4.2. Euler séta irányítatlan gráfban.

A stratégia ugyanaz, mint az irányított gráfnál, de a séta során az élek törlése nem egyszerű. Egy lehetőség az, ha a szomszédokat set-ben tároljuk. De ekkor az algoritmus nem lesz lineáris idejű.

```
typedef
set<int> Graf[maxN];
int n;//pontok száma
Graf G;

void EulerSeta(Graf G, int a, vector<int>& Seta){
  //minden p-re Fok() páros, vagy
  //Fok(a) és Fok(b!=a) páratlan, a többi páros
  stack<int> V;//a vakséta
  V.push(a);
  while(!V.empty()){
    int p=V.top();
    if(G[p].size()==0){
      V.pop();
      Seta.push_back(p);
    }else{
      //set<int>::iterator qt=G[p].begin();
      auto qt=G[p].begin();
      int q=*qt;
      G[p].erase(qt);
      G[q].erase(p);
      V.push(q);
    }
  }
  reverse(Seta.begin(), Seta.end());
}
```

4.3. Euler séta irányítatlan gráfban.

Lineáris idejű megoldáshoz speciális gráfábrázolás szükséges.



25. ábra

```
1  struct El{
2    int p,q;
3    El(){};
4    El(int x, int y){p=x; q=y;}
5  };
6  typedef
7  vector<El> Graf[maxN];
8  int n;//pontok száma
9  Graf G;
10
11 void Beolvas(){
12   //Global:n,G
13   int m,p,q;
14   cin>>n>>m;
15   for (int i=0; i<m; i++){
16     cin>>p>>q;
17     int pf=G[p].size();
18     int qf=G[q].size();
19     G[p].push_back(El(q,qf));
20     G[q].push_back(El(p,pf));
21   }
22 }
```

```
1  void EulerSeta(Graf G, vector<int>& Seta){
2    stack<int> V;//a vakséta
3    int Van[n+1];
4    for(int i=1;i<=n; i++) Van[i]=0;
5    V.push(1);
6    while(!V.empty()){
7      int p=V.top();
8      int i=Van[p];
9      while(i<G[p].size() && G[p][i].p==0) i++;
10     if(i==G[p].size()){//nincs p-ből él
11       V.pop();
12       Seta.push_back(p);
13     }else{
14       El eq=G[p][i];
15       Van[p]=i+1;
16       G[eq.p][eq.q].p=0;
17       V.push(eq.p);
18     }
19   }
20   reverse(Seta.begin(), Seta.end());
21 }
```

4.4. Mélységi bejárás irányítatlan gráfban

4.7. tétel. *Ha* `G` *irányítatlan gráf, akkor bármely mélységi keresésre minden* `p` $-$ `q` *él vagy fa-él, vagy vissza-él.*

Bizonyítás

Tfh. `p`-t előbb értük el, mint `q`-t. Mivel `q` $\in$ `KiEl(G, p)`, ezért valamikor végrehajtódik a `p` $\rightarrow$ `q` él vizsgálta, és ekkor `Szin(p) = Szurke`. Milyen lehet `q` színe ekkor?

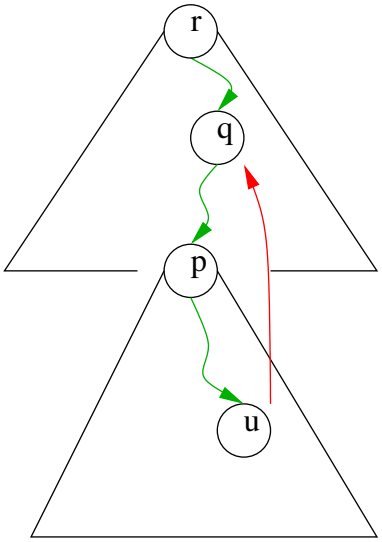
1. `Szin(q) = Feher`: Ekkor `(p, q)` faél lesz.
2. `Szin(q) = Szurke`: Ekkor `(p, q)` visszaél lesz.
3. `Szin(q) = Fekete`: Nem lehet, mert ez azt jelentené, hogy `q`-t már korábban elértük és az összes belőle kiinduló élet, így p-t is vizsgáltuk.

4.4.1. Körben lévő pontok irányítatlan gráfban

Ha a mélységi bejárás során `p` $\rightarrow$ `q` visszaél, akkor a mélységi feszítőfában az úton `p`-től `q`-ig minden pont körben van. Minden körben lévő pont meghatározható ilyen módon, de ennek az algoritmusnak a futási ideje legrosszabb esetben négyzetes.

Állítás

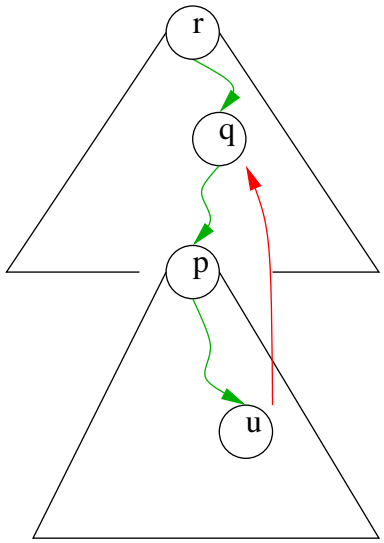
Egy irányítatlan gráfban egy `p` pont akkor és csak akkor van körben, ha a mélységi feszítőfában van olyan `q`, hogy `q`-ből van olyan `q` $\rightarrow$ `u` visszaél, hogy `p` leszármazottja `u`-nak a mélységi feszítőfában. Hatékony (lineáris idejű) algoritmust kaphatunk az u.n. L függvény alkalmazásával.



26. ábra

4.4.2. L függvény

Legyen $G = (V, E)$ irányítatlan gráf, és tekintsük egy r-gyökerű MFF mélységi feszítőfáját. Legyen $D(p)$ a p elérési ideje a mélységi bejárás során. Definiáljuk az $L : V \to \mathbb{N}$ függvényt a következőképpen:



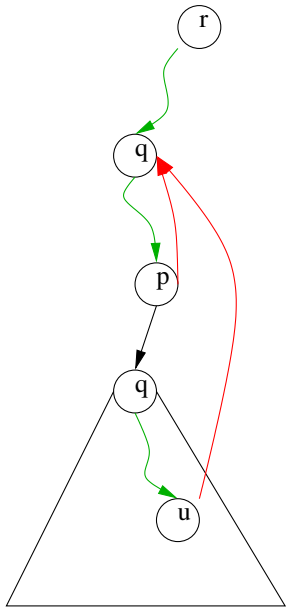
27. ábra

$$L(p) = \min \begin{cases} D(p) \\ D(q) \ : \ \text{van olyan } u \text{ leszármazottja } p\text{-nek, hogy } u \to q \text{ visszaél} \end{cases}$$

Állítás

Az L függvény kiszámítható az élek számával arányos időben (O(|E|)) a mélységi bejárás során.

$$L(p) = \min_{p \to q} \begin{cases} D(p) & \\ D(q) & : \quad p \to q \text{ visszaél} \\ L(q) & : \quad p \to q \text{ faél} \end{cases}$$



28. ábra

```
1  void MelyBejar(int p){
2    //Globális: G, ido, D,Szin,L
3    D[p]=++ido;
4    L[p]=ido;
5    Szin[p]=Szurke;
6    for (int q:G[p])
7    if (Szin[q]==Feher){//p->q faél
8      FeFa[q]=p;
9      MelyBejar(q);
10     L[p]=min(L[p],L[q]);
11   }else
12   if(Szin[q]==Szurke && FeFa[p]!=q && D[q]<L[p])//visszaél
13     L[p]=D[q];
14     Szin[p]=Fekete;
15 }
```

Állítás

Irányítatlan gráf bármely `p` pontja akkor és csak akkor van benne legalább egy körben, ha `L(p) < D(p))`.

```
1  void Korben(Graf& G, vector<int>& KB){
2    for(int x=1;x<=n;x++){
3      Szin[x]=Feher; D[x]=0; FeFa[x]=0;
4    }
5    ido=0;
6    for(int x=1;x<=n;x++)
7      if(D[x]==0) MelyBejar(x);
8    for(int x=1;x<=n;x++)
9      if(L[x]<D[x]) KB.push_back(x);
10 }
```

4.5. Feladat: Vidámpark

A család vidámparkba megy. A vidámpark N részlegből áll, beleértve a bejáratot, amely egyúttal a kijárat is. A részlegeket két-irányban járható útszakaszok kötik össze. Vásárolható olyan kedvezményes jegy, amely esetén minden részleget csak egyszer érintheti a látogató. Kiválasztottak egy részleget, amelyet mindenképpen meg akarnak látogatni. A pénztárnál biztosították a családot, hogy kedvezményes jeggyel is meglátogatható a kiválasztott részleg. Meg kell adni az oda és a visszavezető útvonalat!

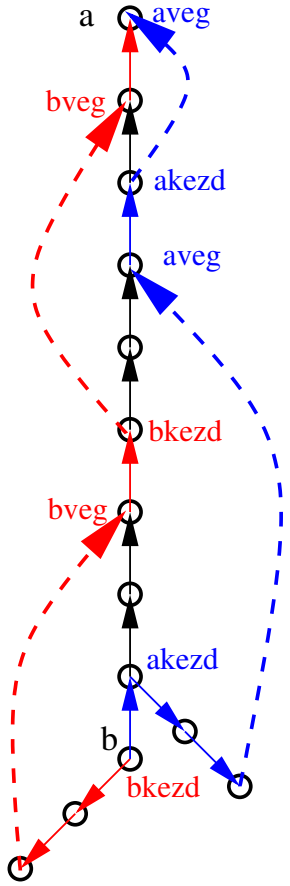
Bemenet

A standard bemenet első sorában a részlegek N ($2 \leq N \leq 100\,000$) száma és a közvetlen útszakaszok M ($2 \leq M \leq 500\,000$) száma, és a kiválasztott részleg R ($1 < R \leq N$) sorszáma van. A bejárat/kijárat sorszáma 1.

Kimenet

A standard kimenet első sorába az oda, a második sorába visszavezető útvonalat kell írni!

Megoldás: keressünk az ábrán látható alternáló utakkal két útvonalat .



29. ábra. Alternáló útvonalak

```
1  typedef
2  vector<int> Graf[maxN];
3  Graf G;
4  int n;//pontok száma
5  int r;//a kiválasztott részleg
6  enum Paletta {Feher, Szurke, Fekete};
7  Paletta Szin[maxN];
8  int D[maxN];//elérési idő
9  int L[maxN];
10 int FeFa[maxN];//feszítőfa
11 int Vissza[maxN];//vissza-él
12 int ido;
13
14 void MelyBejar(int p){
15   //Globális: G, ido, Szin, FeFa, L, Vissza
16   D[p]=++ido;
17   L[p]=ido;
18   Szin[p]=Szurke;
19   for (int q:G[p])
20   if (Szin[q]==Feher){//p->q faél
21     FeFa[q]=p;
22     MelyBejar(q);
23     if(L[q]<L[p]){
24       L[p]=L[q];
25       Vissza[p]=q;
26     }
27   }else if(Szin[q]==Szurke && FeFa[p]!=q && D[q]<L[p]){//visszaél
28     L[p]=D[q];
29     Vissza[p]=q;
30   }
31   Szin[p]=Fekete;
32 }
33 //
34 void Beolvas(int& n){
35   int m,x,y;
36   cin>>n>>m>>r;
37   for(int i=0;i<m;i++){
38     cin>>x>>y;
39     G[x].push_back(y);
```

```
    G[y].push_back(x);
  }
}
void KorKeres(Graf& G, int a, int b, vector<int>& Leut, vector<int
  int FoUt[n+1];
  int Fel[maxN];
  for(int x=1;x<=n;x++){
    Szin[x]=Feher; D[x]=0; FeFa[x]=0; FoUt[x]=0;
    Vissza[x]=0;
  }
  ido=0;
  int akezd=0,aveg=0,bkezd=0,bveg=0;
  MelyBejar(a);

  int x=b;
  while(x!=a){FoUt[FeFa[x]]=x; x=FeFa[x];}
  bool lefel=true;
  Leut.pb(b); //Felut.pb(b);

  while(aveg!=a && bveg !=a){
    if(lefel){
      bkezd= (bveg==0) ? b : bveg;
      int l= (aveg==0) ? L[b] : L[FoUt[aveg]];
      while(L[bkezd]!=l){
        Felut.pb(bkezd); bkezd=FeFa[bkezd];
      }
      Felut.pb(bkezd); bveg=Vissza[bkezd];
      while(bveg>0 && FoUt[bveg]==0){
        Felut.pb(bveg); bveg=Vissza[bveg];
      }
    }else{
      akezd= (aveg==0) ? FeFa[b] : aveg;
      int l= (bveg==0) ? L[b] : L[FoUt[bveg]];
      while(L[akezd]!=l){
        Felut.pb(akezd); akezd=FeFa[akezd];
      }
      Felut.pb(akezd); aveg=Vissza[akezd];
      while(FoUt[aveg]==0){
        Felut.pb(aveg); aveg=Vissza[aveg];
```

```
        }
      }
      lefel=!lefel;
    }
    while(aveg>0){Felut.pb(aveg); aveg=FeFa[aveg];}
    while(bveg>0){Leut.pb(bveg); bveg=FeFa[bveg];}
}
int main (){
    Beolvas(n);
    vector<int> oda;
    vector<int> vissza;
    KorKeres(G, 1, r, oda, vissza);

    for(int i=oda.size()-1;i>=0;i--) cout<<oda[i]<<" "; cout<<endl;
    for(int x:vissza) cout<<x<<" "; cout<<endl;
}
```

5. Feladat 1:Ládák sorba-rendezése, https:/mester.inf.elte.hu/ Mélységi/30.

Egy raktárban N db láda van egy sorban, balról jobbra 1-től N-ig sorszámozva. A ládákat el akar-ják szállítani, ezért mindegyikre rá van írva, hogy melyik városba kell vinni. A raktárban éppen annyi hely van, hogy a ládák elférjenek, de van még egy ládányi hely ideiglenes tárolásra (az ábrán [X]-szel jelölt rész). Kezdetben az i. láda az i. ládahelyen áll, az ideiglenes tárolóhely sorszáma pedig a 0. Mivel a kamion, ami a ládákat elszállítja, először az 1., majd a 2., stb. sorszámú városokba akar menni, és a ládákat csak az ábrán nyíllal jelölt irányból lehet a kamionra pakolni, ezért előzetesen egy targoncának el kell rendeznie a ládákat úgy, hogy bal oldalon legyen az összes olyan, amit az 1. városba, majd amit a 2. városba, stb. kell vinni. A targonca egyszerre mindig csak egy ládát rakhat át egy üres helyre. Mivel a ládák nagyon nehezek, ezért a targoncának az átrendezést a lehető legkevesebb ládaátrakással kell megoldania. Készíts programot, amely kiszámítja a sorba rakáshoz szükséges minimális ládamozgatások számát, és megad egy lehetséges mozgatási sorrendet!

Bemenet

Bemenet A standard bemenet első sorában a ládák száma N ($1 < N \leq 10\,000$) és a városok V ($1 \leq S_i \leq V$) van. A második sorban és a városok ($1 \leq V \leq 200$) db szám van, az i. szám, annak a városnak a sorszáma, ahová az i. sorszámú ládát szállítani kell

Kimenet

Kimenet A standard kimenet első sorába azt az M számot kell írni, ami a minimálisan szükséges mozgatások száma, amellyel a ládák sorba rakhatóak! A következő M sor egy lehetséges minimális mozgatási sorrendet adjon meg: a sorok mindegyike egy számpárt tartalmazzon egy szóközzel elválasztva! Minden i j számpár az i. helyen lévő láda mozgatását jelenti a j-edik helyre.

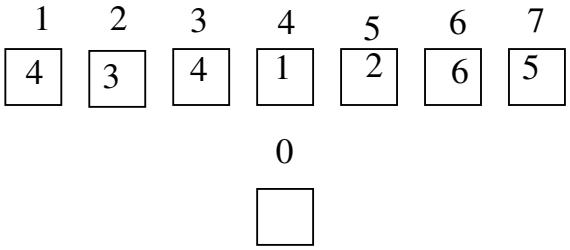
Példa

Bemenet

```
7 6
4 3 4 1 2 6 5
```

Kimenet

```
9
2 0
5 2
1 5
4 1
3 4
0 3
6 0
7 6
0 7
```



30. ábra

Korlátok

Korlátok $1 < N \leq 10\,000$
$1 \leq V \leq 200$
Időlimit: 0.1 másodperc
Memórialimit: 64 MB.

6. Feladat 2:Kritikus csomópontok, https:/mester.inf.elte.hu/ Mélységi/27.

Minden számítógépes hálózat csomópontokból és bizonyos csomópontpárok között kiépített közvetlen kétirányú adatátvitelt biztosító kommunikációs vonalakból épül fel. Adott `A` és `B` csomópont esetén az olyan `P` csomópontot ($P \neq A$ és $P \neq B$), amely meghibásodása esetén nem lehet adatátvitel A és B között, $A - B$-kritikus csomópontnak nevezzük. Írj programot, amely adott `A` és `B` csomópontra kiszámítja a hálózat összes $A - B$-kritikus csomó-pontját!

Bemenet

Bemenet A standard bemenet első sorában a csomópontok száma ($1 < N \leq 20\,000$), azon csomópontpárok száma ($1 < M \leq 200\,000$), amelyek között közvetlen adatátvitel van kiépítve, valamint az `A` és a `B` csomópont sorszáma van. A további `M` sor mindegyike egy `u v` számpárt tartalmaz ($1 \leq u \neq v \leq N$), amely azt jelenti, hogy az `u` és `v` csomópont között közvetlen kétirányú adatátviteli vonal van kiépítve. Bármely két csomópont között legfeljebb egy közvetlen vonal van.

Kimenet

Kimenet A standard kimenet első sorába az $A - B$-kritikus csomópontok `K` számát kell írni! A második sor pontosan `K` egész számot tartalmazzon, az $A - B$-kritikus csomópontokat, tetszőleges sorrendben! Ha nem lehetséges adat-átvitel az `A` és a `B` csomópont között, akkor az első és egyetlen sorba a -1 értéket kell kiírni!

Példa

Bemenet

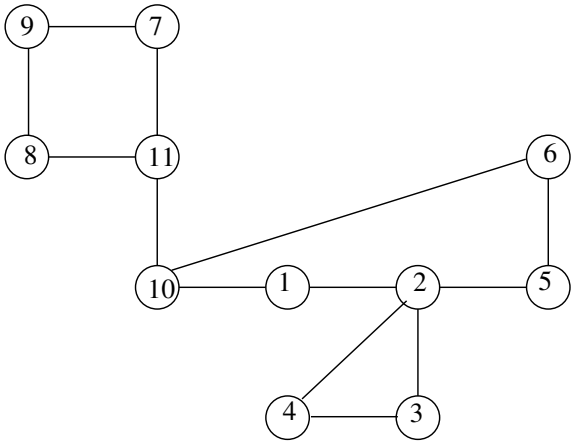
```
11 13 9 5
9 7
9 8
7 11
8 11
11 10
10 1
10 6
1 2
2 3
3 4
4 2
2 5
5 6
```

Kimenet

```
2
10 11
```

Korlátok

Korlátok $1 < N \leq 20\,000$
$1 \leq M \leq 200\,000$
Időlimit: `0.1` másodperc
Memórialimit: `32` MB.



31. ábra